Schema-Driven Information Extraction Beyond the Sentence: Event Extraction, Multilingual Training, and Document-Level Global Decoding for GLiNER2

William Roe[1] (whr778@gmail.com), **Gemini**[2], and **Claude**[3] (noreply@anthropic.com)

[1] Project author and maintainer · [2] AI assistant (Google, Gemini) — diagnosis, architecture, experiments, and drafting · [3] AI assistant (Anthropic, Claude Opus 5) — design, implementation, and drafting

Revision of 2026-11-05 (original results locked 2026-08-19; §4 and §12 updated to cross-reference the boundary-head event record architecture, relational junction layer, and coreference clustering in `PAPER_3_DOCUMENT_EVENTS_AND_COREF.md`). The from-encoder / long-context mmBERT results and the head-initialization finding (§10.5), a broad-data head-init A/B reported as a negative result (§10.6), and an mmBERT data-scaling curve (§10.7) are complete; the §10.1–10.4 tables are the fastino DeBERTa-v3 baselines. All results were re-checked against a split-contamination audit and the audit is reported in full in §7.1, including the one residual it could not remove.

Abstract

GLiNER2 is a single encoder model that performs named-entity recognition, text classification, relation extraction, and hierarchical structured extraction from a schema prompt in one forward pass. This work extends it along four axes. First, we add **event extraction** — ACE-style triggers with typed, multi-valued arguments — as a first-class task threaded through the schema, processor, inference engine, API, and training data model. Second, we make the model **encoder-agnostic and trainable from scratch on any Hugging Face backbone** (e.g. the multilingual, long-context `jhu-clsp/mmBERT-base`), with training infrastructure for Apple MPS, multi-GPU (DDP and DataParallel), gradient checkpointing, sliding-window chunking, and mid-run resume. Third, we build an **evaluation methodology** with strict and relaxed regimes, multilingual (language-ID-driven) blind testing, fine-grained "fair" error analysis after Ortmann (2022), and post-training threshold calibration. Fourth, we address the **document-level** setting — where an event's arguments are dispersed across a document longer than the encoder's window. For the short-context (512-token) DeBERTa-v3 checkpoints we add an opt-in, OneIE-style (Lin et al., 2020) **global graph decoder** that reconnects events across overlapping windows; on a long-context backbone such as **mmBERT** (8192 tokens) the model fits whole documents in one pass and captures the same events *natively*, so global decoding is off by default and reserved for short-context encoders. A pipeline of ~30 corpus converters normalizes public NER, relation, classification, and event datasets into a single JSONL format. All additions are opt-in and default-off, preserving the base model's behavior.

---

1. Introduction

Information extraction (IE) is usually split across task-specific models and sentence-bounded assumptions. GLiNER2 collapses entities, classification, and structured extraction into one schema-conditioned encoder. Two gaps motivated this work:

1. **Task coverage.** GLiNER2 lacked event extraction — the task of detecting an event trigger and filling its typed argument roles — which is central to converting narrative text into structured records.
2. **Scale and locality.** The model and its training/evaluation code assumed short, single-window inputs. Document-level event corpora (e.g. WikiEvents; Li et al., 2021) violate this: arguments routinely sit several sentences from their trigger, and documents exceed the encoder's position limit.

We close both while keeping the base package's inference behavior unchanged: new capabilities are additive and off by default.

2. Background: the GLiNER2 baseline

GLiNER2 encodes `[schema tokens] + [text]`, computes span representations, and scores candidate spans against schema-derived label embeddings. A fluent `create_schema()` builder composes tasks; a single `extract(text, schema)` call returns entities, classifications, structures, and relations. The base models (`fastino/gliner2-base-v1`, `-large-v1`) use a DeBERTa-v3 encoder (512-position cap). Everything below is built on this substrate.

3. Contributions at a glance

- **Event extraction** as an additional first-class task — alongside NER, text classification, relation extraction, and hierarchical structured extraction — consistent across all layers (§4).
- **Encoder-agnostic loading +** `from_encoder()` to train from a raw backbone, and infrastructure for MPS / DDP / DataParallel / gradient checkpointing / sliding-window chunking / checkpoint-resume (§5).
- **Configurable structure losses** (BCE, positive-weighted BCE, focal, asymmetric, Dice, BCE+Dice) with independent per-task (event) overrides (§6).
- **A ~30-converter data pipeline** normalizing public corpora to UTF-8 NFKC JSONL (§7).
- **Evaluation methodology**: strict/relaxed regimes; per-language blind test with language ID; multilingual stopword-aware relaxed matching; CJK tokenization; fine-grained fair error analysis (Ortmann, 2022); post-training threshold calibration (§8).
- **Document-level global event decoding** (OneIE-style) as an opt-in mode on the inference API, the eval config, and a CLI (§9).

4. Event extraction as a first-class task

Events are modeled as multi-field structures: each event type expands to a schema row `[trigger, role_1, …, role_k]`. The feature spans five classes across five files (`Schema`, `SchemaInput`, `SchemaTransformer`, the `GLiNER2` engine, and `SchemaAPI` / `GLiNER2API`), plus the training data model:

- **Schema** — `Schema.events({event_type: [roles]})` (also a richer form with descriptions and per-type trigger/argument thresholds), normalized into `_event_metadata` / `_event_order` and wired through `build()` / `from_dict()` / `from_pydantic()`.
- **Processor** — a `[V]` trigger token; `_process_events()` handles both the inference (schema-only) and training (list-of-mentions) shapes, unioning roles across occurrences.
- **Engine** — `_extract_events()` decodes the top trigger span and all role-argument spans per predicted mention, bucketed under `event_extraction`; public `extract_events()` / `batch_extract_events()`.
- **Training data** — `Event` / `EventArgument` dataclasses that validate every trigger and argument surface appears verbatim in the source text; wired through `validate()` / `sanitize()` / `to_dict()` / `from_dict()`.

A note on tunable event supervision. On the span architecture described here, a dedicated event loss path (`event_struct_loss` / `event_struct_pos_weight`, §6) allowed event supervision to be weighted independently of the other structure tasks. That path did not survive the port to the successor *boundary* architecture, where the loss decomposes by mechanism (span start/end, mention/pair scoring) rather than by task, so event supervision is not independently tunable there. We record this because the two architectures are easy to conflate when reading loss-configuration options: §6 describes the span model.

5. Training from arbitrary encoders and at scale

Encoder-agnostic loading. `from_pretrained` resolves the input-embedding parameter dynamically (via `get_input_embeddings()`), calls `tie_weights()` after resize/load for encoders that tie embeddings (e.g. ModernBERT/mmBERT), and auto-selects device (CUDA → MPS → CPU). A new `from_encoder()` classmethod bootstraps a fresh GLiNER2 (pretrained encoder + randomly-initialized task heads) from a raw HF backbone, enabling training from scratch on `jhu-clsp/mmBERT-base` and similar. Because mmBERT/ModernBERT carry an 8192-token context, a model bootstrapped this way fits nearly all document-level event corpora in one forward pass (§9.5) — the native alternative to the windowing that short-context (512-token) encoders require.

Infrastructure. The trainer adds: an MPS device tier (with a cuDNN-SDPA workaround for long variable-length sequences); multi-GPU via torchrun **DDP** (rank-0 eval + broadcast early-stop) and opt-in `nn.DataParallel` (with an autocast-preserving replica wrapper); **gradient checkpointing**; **sliding-window chunking** of long training records into overlapping subword windows; **checkpoint-restart** (optimizer/scheduler/RNG state for true mid-run resume); generalized OOM handling and cross-backend cache management; the modern `torch.amp` API; and pre-training ETA estimation.

6. Configurable structure losses

`ExtractorConfig.struct_loss` selects among `bce`, `bce_posweight`, `focal`, `asl` (asymmetric loss), `dice`, and `bce_dice`, with per-variant hyperparameters. Region-based Dice bypasses the random-negative-masking path the pointwise variants use. Crucially, `event_struct_loss` / `event_struct_pos_weight` override the loss **for events only**, so entity and relation supervision can be tuned separately from the sparser, harder event supervision. Loss is tracked as four components (`classification` / `structure` / `event_structure` / `count`). Each loss variant ships with a matching YAML config for controlled comparison.

7. Data pipeline

`tools/data/` contains ~30 converters that normalize public corpora into a single GLiNER2 JSONL schema, all routed through a shared writer that emits UTF-8, NFKC-normalized records with stray Unicode line separators stripped (so records never fragment across lines). Coverage includes:

- **NER**: NuNER, Pile-NER, PubMed, BC4CHEMD, BC5CDR, FiNER-ORD, Stockmark (ja), KazNERD, KLUE (ko), SciERC, BioRED, and generic HF token-NER.
- **Relations**: DocRED, Re-DocRED, SciERC, BioRED, sentence-REx, bio-NER-relations.
- **Classification**: gliclass-logic, Scientific-text, paraloq schema extraction.
- **Events**: ACE 2005 (with NAM/NOM/PRO mention-filter variants), WikiEvents, DocEE, RAMS, MAVEN, CASIE, CMNEE (zh).

Converters are config-driven (e.g. entity mention-type filtering, label roll-up/remap) and stratify train/val/test splits deterministically.

7.1 Split integrity: a contamination found mid-project, and what it changed

Every number in this paper was re-checked against a split-contamination audit completed 2026-08-18, and the audit is reported here rather than quietly fixed. We think other groups will find the shape of the failure more useful than the specific counts: it was invisible to ordinary review, it was introduced by a plausible-looking line of code, and the first count we took of it was itself wrong.

The defect. The split writer drew a random number **per row** rather than per document. Any document emitted more than once — which is normal in these corpora, where one document yields several task records — had its copies scattered independently across train, validation and test. Nothing crashed and no metric looked anomalous; the leak is silent by construction.

Scope. A repo-wide gate over all corpora on disk found **45 of them shipping overlapping splits**, repaired by dropping **21,553 records of 2.3M (0.94%)** under the precedence *test* > *val* > *train*, so the blind test keeps every document it has and the training side gives up the duplicate. Worst affected were corpora unrelated to this paper (`gliclass_logic` 25.7%, `knowledgator_gliner` 15.6%, `klue_re` 11.3%). An earlier count of "~15 corpora" was taken from a truncated scan and was wrong — worth recording, because an under-count of a contamination is more dangerous than no count at all.

Two tool defects had made some corpora *permanently* unrepairable and are the reason the audit had to be run over everything rather than sampled. The deduplicator derived split paths as `.val.jsonl` only, so corpora naming that split `dev` — including **WikiEvents and RAMS**, both used here — were silently skipped while the tool reported success on the pairs it did check. Separately, a require-all-three-splits check rejected any corpus missing a split outright.

Effect on this paper's results. Of the corpora behind the results below:

corpus	pre-repair overlap	affects
RAMS	**none** — canonical 7,329 / 924 / 871 on disk, gate passes	§10.5–10.7 unaffected
CASIE	**none**	§10.3 unaffected
RE-DocRED	1 document, train ∩ val (0.2% of val); **test untouched**	§10.4 blind test clean; checkpoint selection only
WikiEvents	1 document train ∩ test (5.0% of the 20-document test set), and 1 document train ∩ dev	§10.1, §10.2, §10.3 WikiEvents row

That RAMS is clean matters most: it carries §10.5–10.7, the head-initialization finding, which is this paper's principal empirical contribution.

What we did about WikiEvents. The contaminated document was recovered by regenerating the pre-repair split from the converter (206 records, 202 distinct documents — matching the archived pre-repair count exactly, confirming the converter had not drifted) and diffing document keys against the repaired split. Every affected result was then **re-scored on the 19 uncontaminated test documents, same checkpoints, identical eval configuration**. The result:

WikiEvents blind test (base-v1)	as published	re-scored, 20 docs	**re-scored, clean 19**
entity strict F1	0.764	0.7725	**0.7718**
event_type strict F1	0.953	0.9791	**0.9791**
event_trigger strict F1	0.551	0.5417	**0.5417**
event_argument strict F1	0.136	0.1366	**0.1366**
event strict F1	0.378	0.3845	**0.3845**

Every event metric is unchanged to four decimal places, and entity F1 moves by −0.0007. The reason is specific rather than lucky: the contaminated document is a definitional passage carrying **13 entity mentions and zero events**, so it could not have influenced an event metric. `gliner2-large-v1` behaves identically (entity 0.7949 → 0.7946, all event metrics unchanged), and the §10.2 windowing ablation reproduces on the clean set with its conclusion intact (§10.2).

The gap between the "as published" and "re-scored" columns is **not** contamination — the test split is byte-identical. It is eval-code drift over the intervening months (chiefly event-type decoding and the fair-error analyser). We report both columns rather than silently restating, and the headline `event_argument` figure is unchanged either way.

What we could not fix by re-scoring, stated plainly. The second WikiEvents overlap put one document in both train and dev, and that document *does* carry events — 23 of dev's 345 (6.7%), 21 of 428 arguments (4.9%). Checkpoints were selected on dev event F1, so **selection saw a mildly optimistic signal**. Removing that residual requires retraining on the repaired split, not re-scoring, and we have not done it; the reported WikiEvents figures should be read as carrying that caveat. It does not touch RAMS, CASIE or the RE-DocRED test split.

Now gated. Within-split overlap is checked automatically before every training run (`tools/data/check_leakage.py --config`, wired into `tools/train/train.py`), the deduplicator handles `dev`-named and missing splits, and each hosted corpus carries its own pre-repair overlap counts on its dataset card so the contamination history travels with the data.

7.2 Real text with synthetic labels vs synthetic text with synthetic labels

Both halves of a training corpus can be synthetic — the TEXT, and the ANNOTATION over it. They are separable, and we measured the difference. **The two axes disagree, and treating "synthetic data" as one variable produces the wrong decision.**

Axis 1 — preservation of prior capability. A 2×2 fine-tuning study over `base-v1`, scored on a common held-out set (`pile_ner_def` + `knowledgator_gliner` val, stride-6 slice, best-vs-best, entity strict micro F1):

arm	text	annotation	entity strict F1	Δ vs base
base-v1	—	—	0.5322	—
`synthetic_haiku45_5k`	**synthetic**	synthetic	**0.4099**	−0.1223 (−23.0%)
`cc_news_haiku45`	**real**	synthetic	0.3596	−0.1726 (−32.4%)
real + synthetic mix	both	synthetic	0.3267	−0.2055 (**−38.6%**)

Real text forgot MORE, not less — the opposite of the pre-registered prediction, and the mix was worst of all. Partly confounded by volume (10,960 optimizer steps against ~5,080), but the confound does not explain it away: step-matched at epoch 5 (~5,480 steps) real text still preserves 0.3653 against synthetic's 0.4099.

Axis 2 — transfer to real deployment text. Here the ordering reverses, and far more sharply:

system	training positives	in-distribution	on real news
stage-0 relevance gate	**99.9% synthetic**	F1 **1.0000**	admits **0 of 71** articles; 0 of 590 SMS
EKF front end	**71.4% synthetic** English trigger+argument	right trigger, right bound argument on its own corpus	forms events on FEWER real wire-copy windows than the model it replaced
casualty extractor	100% synthetic English	strong on its own corpus	`location` holds a digit **78.2%** of the time on Turkish news vs 5.8% on English (p = 1.5e-39)

A gate scoring F1 = 1.0000 that admits nothing real is not a good gate; it has a trivially perfect false-positive rate because it never fires. That failure was originally attributed to truncation and CJK handling, and the corrected root cause is the provenance split: **the positive class was synthetic and the negative class was real**, so the model learned to separate registers rather than relevance.

The synthesis. Synthetic text is the safer regulariser and the worse teacher of the deployment distribution. Real text with synthetic labels costs more preservation and buys transfer that synthetic text does not provide at any volume. Which matters depends on whether the model must keep prior capability (use replay — 5–10% is the minimum that prevents catastrophic forgetting, ~30% is better) or must work on real input (use real text, and check the class balance for a provenance confound before believing any in-distribution score).

A corollary we now design around. When positives and negatives come from different provenance, an in-distribution metric measures provenance, not the task. Balance the provenance across classes, or report the score as uninterpretable.

7.3 Measure what the base already models, before buying supervision

Supervised data does not rescue a language or a register the encoder cannot model, and forcing it distorts the representation for the ones it modelled well. We therefore measure masked-LM pseudo-perplexity per language and per domain on held-out text drawn from the corpora we intend to train on, and treat it as a gate on the purchase rather than a post-hoc diagnostic (`tools/train/base_model_perplexity.py`).

mmBERT-base, 120 held-out documents per cell:

language	pseudo-PPL	tokens/char		English domain	pseudo-PPL
Kazakh	2.26	0.465		financial filings	3.13
Turkish	**2.46**	0.269		biomedical	4.15
English	4.93	0.254		**news (real)**	**4.67**
Chinese	**5.90**	0.318		news (synthetic)	5.42
Japanese	10.36	0.603		scientific	8.56
African langs (MasakhaNER)	**114.11**	0.366		**short-form messages**	**29.52**

Perplexity is per-token, so `tokens/char` is reported beside it: cross-encoder comparison is only meaningful when the tokenizer is shared, and a language can look cheap per token while costing three times the tokens.

Two decisions came out of this table. Turkish at 2.46 and Chinese at 5.90 are firmly modelled, so the multilingual casualty purchase was sound — the labels would take. The MasakhaNER row at 114.11 is the opposite case: adding supervision there would be the wrong lever, and continued unsupervised pretraining the right one.

The domain axis re-explained a failure we had already misdiagnosed. Our stage-0 relevance gate admitted 0 of 590 disaster-related SMS messages. We attributed that entirely to a provenance confound in its training set — a synthetic positive class against a real negative class — which was real but not complete: the encoder itself sits at 29.52 on that register against 4.67 on news. Part of that failure is representational, and no quantity of supervised SMS labels closes it without continued pretraining.

The converse is equally useful and points the other way for our main line: mmBERT models real news as well as anything we measured, so the EKF front end's real-news weakness is **not** representational. It is supervised signal, and continued pretraining would not have helped.

7.4 A language-balanced corpus, and two constraints that pull against each other

`casualty_ml` is 62,703 `casualty_report` records balanced at 20,901 per language across English, Turkish and Simplified Chinese, all REAL news. Building it surfaced a tension worth stating because the obvious construction satisfies only one side.

The base trained on `docee.train`, and 6,247 of the English casualty rows are derived from that text. They are legitimate training data — the labels are new — but scoring them is not a blind test. Pinning them to train shrinks the English evaluation pool, and letting a fixed 80/10/10 ratio absorb the loss yields a test set of **25.1% English against 37.5% Turkish**, which lets a model strong in two languages post a good aggregate. Sizing val and test *equally per language by construction* (1,400 each, drawn only from base-unseen rows) and letting train take the remainder satisfies both: 0 base-trained rows in val or test, and 33.0/33.5/33.5 in every split.

Blind-test integrity and evaluation balance are separate properties, and enforcing one by ratio silently trades away the other.

8. Evaluation methodology

Strict and relaxed regimes. Every category (entity, relation, classification, and the event sub-metrics `event_type`, `event_trigger`, `event_argument`, and combined `event`) is scored twice: *strict* (exact surface, one-to-one) and *relaxed* (type exact + surface overlap, stopword-aware and normalized, so relaxed never scores below strict). Details in `METRICS.md`.

Multilingual. A per-language blind test uses language ID to bucket records, reports per-language then combined, and a multilingual stopword set (auto-detected from corpus language codes) drives relaxed matching; CJK text is tokenized per-character so Asian-language spans are scored correctly.

Fine-grained fair error analysis (Ortmann, 2022). Standard P/R/F1 double-penalize a near-miss (a right-surface/wrong-label or right-label/wrong- boundary span counts as both a false positive and a false negative). We add an additive diagnostic that matches predictions to gold *across labels* and tags each with one typed error — `COR`, `LE` (labeling), `BES` / `BEL` / `BEO` (boundary sub-types), `LBE`, `FP`, `FN` — records label confusions, and reports a *fair* P/R/F1. We follow the **reference FairEval tool's default weights**, which are the paper's Eq. 6/7 rather than its plainer Eq. 5: a near-miss is charged once instead of twice, *and* a boundary error earns partial true-positive credit, because the system did find the span.

```
LE  = 0.5 FP + 0.5 FN            right span, wrong label — no credit
LBE = 0.5 FP + 0.5 FN            both wrong — no credit
BES = 0.5 TP + 0.5 FN            system span smaller: a recall miss
BEL = 0.5 TP + 0.5 FP            system span larger: a precision miss
BEO = 0.5 TP + 0.25 FP + 0.25 FN
```

Unlike Eq. 5 these move F1 rather than only P and R, because they add TP mass. One consequence for our implementation: the BES/BEL/BEO split is inferred from substring containment rather than character offsets, and under these weights that split now *changes the score* (BES costs recall only, BEL precision only), where under Eq. 5 all three weighed the same.

`eval_<cat>_fair_micro_f1` is emitted for entities and event spans. The trainer's built-in default for `metric_for_best` remains `eval_loss`, but the event A/B and synthetic-fine-tune configs now **select on fair F1**, on the argument that it is more informative than strict micro-F1 for checkpoint choice.

Threshold calibration. After training, `sweep_thresholds` re-scores the validation set over a small grid and picks the decision threshold maximizing a support-weighted micro-F1, writing the choice to the model card and `threshold_sweep.json`.

9. Document-level event extraction via global decoding

9.1 Problem

This problem is specific to **short-context** encoders whose window is smaller than the document; on a long-context backbone (§9.5) it does not arise. The fastino GLiNER2 checkpoints use DeBERTa-v3 (512-position cap), so training uses a 384-word sliding window and trigger↔argument association is **window-bounded**, while WikiEvents records are whole documents (median ~500 words; 50–70% exceed 512 words). Evaluation historically fed whole documents to the model in a single pass — running it out of its trained window and past its position cap. The measured consequence is that **arguments are the bottleneck**: on the recorded WikiEvents blind test, `event_type` strict F1 is 0.82–0.93 but `event_argument` strict F1 is only 0.068 (base) / 0.28 (large).

9.2 Method

We scan the document with **overlapping** windows (window = training `max_len`; overlap = 128 words → step 256, matching training's stride), remap each window's spans to global document offsets, and treat every trigger/argument/entity candidate — each with a confidence — as a node in a document-level graph. A **greedy** substrate clusters event mentions across windows by trigger IoU and unions their arguments

(deduplicating by role and span, pooling confidence). A **beam decoder** then refines the result under global constraints — role validity (hard), single-filler cardinality, a cross-event span-conflict penalty, and a trigger-confidence floor — mirroring the local-scoring-plus-global-decode design of OneIE (Lin et al., 2020). The assembler emits the exact standard `event_extraction` shape, so all downstream formatting and metrics are unchanged.

9.3 Interfaces

The mode is opt-in on three surfaces, all default-off:

- **Inference API**: `batch_extract_long(..., global_decode=True)` / `extract_long(...)`.
- **Evaluation**: `compute_metrics(..., chunk_size=…, global_decode=True)`, driven from the training YAML (`eval.global_decode`, `eval.chunk_size`, `eval.chunk_overlap`); windowing and the decoder are separable knobs so a three-way A/B (whole-doc baseline vs. chunk+simple-merge vs. chunk+beam) is measurable.
- **CLI**: `tools/infer.py --global-decode`.

9.4 Honest limitations

OneIE *learns* its global-feature weights; we have no such training signal, so the beam uses **heuristic/config-set** weights (`GlobalDecodeConfig`). Recall is still bounded by within-window candidate recall — an argument more than ~one window from its trigger is never emitted and cannot be recovered by any post-hoc decode (this would require trigger-anchored windows or a two-stage document reader; see `PAPER_0_FOUNDATION.md sec 9`). Trigger clustering by span overlap can, rarely, merge two adjacent same-type events. The value the beam adds over greedy is cross-event conflict resolution. Correspondingly, the mode is only beneficial when documents exceed the encoder's one-pass window; on a long-context backbone that sees the whole document, chunk- and-reassemble would be strictly worse — hence opt-in, never default.

9.5 The long-context alternative: mmBERT

Global decoding is the *short-context* remedy. The higher-leverage fix is to remove the window bound entirely by training on a **long-context backbone**. The branch's namesake, `jhu-clsp/mmBERT-base` (ModernBERT architecture, 8192-token context; Warner et al., 2024; Marone et al., 2025), fits the large majority of these documents in one forward pass — WikiEvents' median is ~700 subword tokens and its 95th percentile ~2.5k, well under 8192; only the longest tail overflows. A GLiNER2 model bootstrapped from mmBERT via `from_encoder()` and trained with a wide `max_len` therefore sees each trigger together with its scattered arguments, capturing document-level events **natively** — no windowing, no cross-window reassembly, no boundary loss. On such a model global decoding is not merely unnecessary but counter-productive (it would fragment context the encoder could use whole), which is why it is off by default and gated on `chunk_size`.

One residual limit is orthogonal to context length: the span model's count head is a 20-way classifier (`count_pred`), so it emits at most **19 instances per schema type per document** (higher gold counts are clamped to 19 in training), and the native single-pass path hits this ceiling hardest — it has no cross-window union to exceed it. It is a hard cap for high-cardinality tasks such as document-level relations (Re-

DocRED routinely exceeds 19 triples per document). The boundary architecture's continuous per-query count log-rate removes the cap.

The two paths are complementary, selected by the deployment constraint:

Regime	Encoder	Document-level events via
Short context (≤512)	DeBERTa-v3 (fastino base/large/multi)	windowing + global decoding (§9.2)
Long context (8192)	mmBERT / ModernBERT (`from_encoder`)	native single pass; global decode off

The trade-off is compute: 8192-token attention is quadratic in length, which is why the training infrastructure (§5) invests in gradient checkpointing and cross- backend memory management. Global decoding stays the pragmatic choice when a short-context checkpoint must be used as-is, or for the rare document exceeding even the long-context window.

10. Results

Blind test on the WikiEvents held-out set for the **base-v1** run (DeBERTa-v3-base, 15 epochs), evaluated with windowed global decoding (`chunk_size 384` , `chunk_overlap 128` , `global_decode true`). Micro unless noted. **All WikiEvents figures in §10.1–10.3 are on the 19 uncontaminated test documents** (§7.1); the excluded document carries no events, so every event metric is identical on the full 20.

10.1 WikiEvents blind test — base-v1

Category	strict P / R / F1	relaxed F1	fair F1	support
entity	0.727 / 0.823 / **0.772**	0.810	0.809	1589
event_type	1.000 / 0.959 / **0.979**	0.979	—	122
event_trigger	0.495 / 0.598 / **0.542**	0.557	0.564	239
event_argument	0.158 / 0.120 / **0.137**	0.437	0.474	515
event (combined)	0.403 / 0.368 / **0.385**	0.557	—	876

Event *type* detection is nearly saturated (0.979, precision 1.00); *arguments* remain the bottleneck at strict 0.137 but recover strongly under relaxed (0.437) and fair (0.474) — the model often locates the right argument entity with an inexact boundary/surface or under strict's double penalty. The fine-grained error counts bear this out: of the gold arguments the fair analysis scores, COR 175, typed near-misses (LE/LBE/BE*) 46, pure misses (FN) 231, plus 123 false positives.

10.2 Effect of windowing + global decode

Clean 3-point ablation on this base-v1 checkpoint — model fixed, only the eval path varies — over the full 20-document test set:

Eval configuration	event_argument strict F1	arg relaxed	event strict F1
A. whole-doc single pass	0.086	0.393	0.369
B. chunk (overlap 128) + simple merge	**0.144**	0.434	0.380
C. chunk + OneIE global decode (beam)	0.137	0.437	0.385
D. chunk + beam, config fit on val †	0.129	0.402	0.387

† Rows A–C are re-measured on the clean 19-document test set (§7.1). Row D is carried from the original run: reproducing it requires re-fitting `GlobalDecodeConfig` on validation, and the validation split is the one whose residual contamination §7.1 could not remove, so a re-fit would not be cleaner. It is reported for the qualitative point it supports — that fitting the decoder does not change the verdict — not as a directly comparable number.

Windowing is the driver. Chunking eval to match the model's trained window (A→B) lifts argument strict F1 **0.086 → 0.144** (+67% relative), fixing the train/eval mismatch and the 512-token overflow. **The beam adds essentially nothing over a simple chunk-merge here** (B→C): argument strict is marginally *lower* (0.144 → 0.137) while event strict is marginally higher (0.380 → 0.385) — within noise. This is the measured, honest answer to "does the beam earn its keep": on WikiEvents, not appreciably. Its value (cross-event conflict resolution under heuristic weights) is below the noise floor, and the default no-cardinality-cap union slightly over-generates arguments, nudging strict precision down. The `single_filler_roles` config is the lever to recover that; a learned global scorer would be the principled fix.

Fitting the decoder on validation (row D) does not change the verdict. `sweep_global_decode` calibrates the beam's `GlobalDecodeConfig` on the dev set exactly as `sweep_thresholds` calibrates the decision threshold (§8) — a grid over `conflict_penalty` × `min_trigger_conf`, choosing the support-weighted strict micro-F1. The fit moves `conflict_penalty` 0.5 → 1.0, but `min_trigger_conf` is entirely inert (every WikiEvents trigger clears the floor), and on the test set the fitted config lands within noise of the default beam: event strict +0.009 (0.378 → 0.387), argument strict −0.006 (0.136 → 0.129), argument relaxed −0.033. So the flat response is **structural, not a tuning artifact** — the loss is recall-bound (§10.1: 231 FN), which no post-hoc re-ranking reaches. A fully learned OneIE scorer would optimize the same recall-bound dimension (§12).

Decomposing the 0.068 → 0.136 change from the prior baseline (approx., since the prior run's exact config is inferred): ∼+0.018 from 5x more training (prior 3-epoch whole-doc 0.068 → this 15-epoch whole-doc 0.086), ∼+0.057 from windowing (0.086 → 0.143), and ∼−0.007 from the beam vs. simple merge. The takeaway reframes the contribution: **the windowed eval path — not the OneIE decoder — is what recovers the document-level argument bottleneck** on a short-context model; the decoder is an optional, roughly-neutral refinement pending learned weights or cardinality tuning.

10.3 Full event-dataset sweep

Blind-test micro-F1 for every fine-tuned configuration, evaluated with windowed global decoding. `event_argument` is given as strict / relaxed / fair. A dash means the category was not scored for that configuration, generally because the corpus as converted carries no annotation for that layer — RAMS and CMNEE, for instance, supply events without an entity layer.

These rows are reported **as run**. Several converters have changed since (DocEE's splits were rebuilt as recently as 2026-08-19 after upstream contamination was found in them), so the entity/event coverage of a row does not always match what today's converter emits for that corpus — the MAVEN and DocEE rows in particular. They are kept because they are the measurements the models were actually scored on; they should be re-run rather than re-derived if the exact numbers matter.

Config	entity	event_type	event_trigger	event_argument (S / R / Fair)	event	support
`gliner2-base-v1-casie`	0.553	0.928	0.413	0.058 / 0.450 / 0.426	0.207	3454
`gliner2-base-v1-docee`	0.333	—	—	— / — / —	—	—
`gliner2-base-v1-maven`	0.742	—	—	— / — / —	—	—
`gliner2-base-v1-mendeley-ed`	0.741	—	—	— / — / —	—	—
`gliner2-base-v1-rams`	—	0.993	0.935	0.462 / 0.686 / 0.614	0.693	3712
`gliner2-large-v1-casie`	0.591	0.975	0.487	0.173 / 0.549 / 0.515	0.302	3454
`gliner2-large-v1-docee`	0.360	—	—	— / — / —	—	—
`gliner2-large-v1-rams`	—	1.000	0.903	0.444 / 0.697 / 0.627	0.684	3712
`gliner2-large-v1-wikievents` ‡	0.795	0.988	0.574	0.119 / 0.470 / 0.505	0.371	876
`gliner2-multi-v1-cmnee`	—	0.986	0.874	0.221 / 0.709 / 0.671	0.466	27099

‡ Re-measured on the clean 19-document WikiEvents test set (§7.1); its event metrics are identical on the full 20 and entity moves −0.0003. Every other row is on a corpus the contamination audit found clean.

10.4 RE-DocRED relation extraction

Fine-tuning `fastino/gliner2-{base,large}-v1` on RE-DocRED (re-annotated DocRED: document-level NER + relation extraction). Blind-test strict micro-F1; the checkpoint is selected on `eval_relation_strict_micro_f1` with windowed eval (`chunk_size 256` , matching the sliding-window training) so the ~500-word docs do not overflow the 512-position cap. 10 epochs, early stopping (patience 3).

Config	entity	relation	support (ent / rel)
`gliner2-base-v1-redocred`	0.842	0.263	10705 / 17348
`gliner2-large-v1-redocred`	0.860	0.287	10705 / 17348

Document-level strict relation F1 is a hard metric for a span-based extractor (cross-window pairs and coreferent arguments are recall ceilings); the large model leads on both entity and relation F1.

10.5 From-encoder training and the head-initialization bottleneck

We ran the experiment §12 proposed: train events from a *raw* encoder via `from_encoder()` (fresh GLiNER2 heads) on RAMS, on the long-context mmBERT-base backbone (8192, whole document natively; §9.5). As an **encoder-isolation control** we ran the identical recipe on DeBERTa-v3-base — same fresh heads, same loss (`bce_posweight`, `pos_weight 4`), same 15 epochs and argument-strict checkpoint selection — but with DeBERTa's native short-context handling (384-word windows + global decode). Both contrast with fastino `gliner2-base-v1-rams`, whose heads are IE-curriculum-pretrained on ~254K examples (Zaratiana et al., 2025) *before* RAMS fine-tuning. RAMS blind test, strict micro-F1:

Model	Encoder	Heads	event_type	event_trigger	event_arg (S / R)	event
mmBERT-base `from_encoder`	mmBERT (8192, native)	fresh	0.964	0.611	**0.050** / 0.213	0.247
DeBERTa-v3-base `from_encoder` (control)	DeBERTa-v3 (384 window)	fresh	0.981	0.612	**0.042** / 0.204	0.340
`gliner2-base-v1-rams` (fastino)	DeBERTa-v3 (384 window)	**pretrained**	0.993	0.935	**0.462** / 0.686	0.693

The long-context model did not recover the argument bottleneck — and the control says why. Hold the heads fresh and swap the *encoder* (mmBERT ↔ DeBERTa): arguments barely move (0.050 vs 0.042), triggers not at all (0.611 vs 0.612). Hold the *encoder* fixed (DeBERTa-v3) and swap fresh → pretrained heads: arguments jump ~11× (0.042 → 0.462) and triggers 0.612 → 0.935. The RAMS argument gap is therefore dominated by **head initialization** — the IE-curriculum pretraining of the fastino heads — **not** the encoder or the context window. mmBERT's 8192 tokens cannot help because the binding constraint is untrained extraction heads, not window-bound truncation (RAMS documents fit in 512 regardless). This refines §9.5: long context remedies window-bound *recall loss* on genuinely long documents, but it is **orthogonal** to the head-competence bottleneck that dominates argument extraction from a cold start.

What the fastino head-init actually is (verified against Zaratiana et al., 2025; see `FASTINO_GLINER2_TRAINING.md`). The fastino heads were trained **fully supervised** on **254,334 examples** — 53% real text (news/Wikipedia/legal/PubMed/ArXiv) and 47% synthetic, *all* GPT-4o-annotated (LLM knowledge-distillation, not human labels) — for just **5 epochs** with a **differential learning rate** (task heads 2×10^{-5}, encoder 1×10^{-5}, so heads adapt ~2× faster). Crucially, the training tasks were **entities, hierarchical structure extraction, and classification — no events and no relations.** The competence that transfers is the **hierarchical-structure head**: a shared span-matching scorer (bilinear dot-product +

sigmoid), a 20-way instance-count MLP, and occurrence-ID conditioning for per-instance fields. Our event path *reuses those exact heads* (trigger/argument = the span scorer, multiple mentions = the count head, per-event arguments = occurrence-ID conditioning), which is why warm-starting a model that never saw an event still lifts RAMS/WikiEvents arguments — the head-init effect is **structural, not event-specific**.

The implication is constructive: the remedy is to give a from-encoder model the same *scale* of structure-extraction supervision. Because it is the structure head that transfers, the target is **not** ~10^5–10^6 *event* documents but a large **hierarchical-structure / argument** curriculum in the fastino mold (mixed real+synthetic text, LLM-annotated and validated). `tools/data/synthetic/` builds exactly that shape — broad-label, multi-task supervision (entities, relations, document-level events with triggers + arguments, classification, structures), either fully generated or projected as synthetic annotations onto real corpora, at the ~10^5–10^6 scale the fastino heads saw. Head-init pretraining, *then* task fine-tuning, is the path to closing this gap on a long-context backbone.

The WikiEvents from-encoder run (mmBERT, warm-started from the RAMS checkpoint above) reinforces the same point as a **documented negative result**: with only 206 training documents the argument signal never rises above ~0, so argument-strict checkpoint selection is degenerate and the model is weak (entity 0.133, event_type 0.970, event_trigger 0.127, event_argument 0.000). It is reported here but **not released** — 206 documents cannot substitute for head pretraining. Only `whr778/mmbert-base-rams` is published.

Methodology — per-epoch threshold sweep. `bce_posweight` up-weights positive spans, shifting the score distribution so a fixed 0.5 decision threshold no longer reflects the model's operating point: argument-strict F1 *at 0.5* collapses to near-zero noise for every epoch, making fixed-threshold checkpoint selection effectively random. We added `eval.metric_sweep`, which evaluates `metric_for_best` each epoch at the threshold that maximizes it over a small grid (0.1–0.9), so the saved checkpoint is the genuine argument peak at its own calibrated operating point. The end-of-run blind-test sweep keeps the aggregate support-weighted objective (§8) for apples-to-apples comparison with the baselines — which, for argument-sparse categories, *understates* the peak, so the from-encoder argument numbers above are conservative.

10.6 Head-init pretraining: a first broad-data A/B (negative result)

§10.5 argued the remedy for the argument bottleneck is to give a from-encoder model the same IE curriculum the fastino heads saw. We ran a first, deliberately cheap version of that experiment and report it as a **negative result**.

A broad **combined base** — mmBERT-base `from_encoder`, warmed for 2 epochs on our multi-task synthetic corpus (`synthetic_sonnet5`: entities, relations, events, classification, structures), GLiNER multilingual NER, GLiNER multi-task NER, and RAMS document events (~96K records, `eval_loss` checkpoint selection) — was then fine-tuned on WikiEvents under the **identical** 15-epoch, argument-strict recipe used for the RAMS-only base (`whr778/mmbert-base-rams`). Only the base checkpoint differs, isolating the base-data effect. WikiEvents blind test (20 docs; 239 trigger / 515 argument mentions), strict micro-F1:

WikiEvents fine-tune	base	event_type	event_trigger	event_argument (S)
control	RAMS-only (`mmbert-base-rams`)	**0.944**	0.085	0.0046
treatment	broad combined base	0.573	**0.133**	0.0066

The control reproduces the near-floor WikiEvents-from-RAMS result of §10.5 (event_argument ~0, event_type ~0.95, event_trigger ~0.1) within run-to-run noise on 20 documents. **The broad+synthetic base gave no reliable downstream lift.** Argument-strict F1 stays at the floor for both (0.007 vs 0.005; identical 0.004 recall — roughly two correct arguments each, i.e. noise on 20 documents). The trigger edge to the treatment (0.133 vs 0.085) is precision-only on 239 mentions, within run-to-run noise, and the broad base *regressed* event_type (0.573 vs 0.944). For context, the combined base's own RAMS argument head was also weak (arg-strict 0.028) — though under a lighter regime than the 0.050 RAMS-only figure in §10.5 (2 epochs + `eval_loss` selection vs 15 epochs + argument-strict selection), so it is context, not a controlled base-to-base claim; the controlled comparison is the WikiEvents A/B above.

This does **not** refute §10.5 — it shows a *light* broad-data pass is not a substitute for real head-init pretraining. The confounds point the same way: only 2 base epochs, `eval_loss` selection (dominated by the ~77K multilingual-NER records), and argument dilution mean the argument head was never warmed at the $\sim 10^5$–10^6 scale, argument-strict-selected, that the fastino curriculum used. Both combined checkpoints are retained privately (`whr778/mmbert-base-combined`, `whr778/mmbert-base-combined-wikievents`), not released.

Sanity check — the synthetic data itself trains cleanly. To rule out "bad synthetic data" as the cause of the null result above, we fine-tuned the pretrained `fastino/gliner2-base-v1` on `synthetic_sonnet5_1k` *alone* (warm-start, 10 epochs, `eval_loss` selection). On the synthetic held-out split it scores well across all five tasks — strict micro-F1: entity 0.904, relation 0.657, event-type 0.956, event-trigger 0.838, event-argument **0.702** (0.894 relaxed), classification 0.835. So the synthetic corpus is coherent and learnable (in-distribution — this is not a claim about the public benchmarks); the combined-base null result is about the **head-init regime** (2 epochs, `eval_loss` selection, NER dilution), not the data. Checkpoint: `whr778/gliner2-base-v1-synthetic`.

Conversely, the synthetic data cannot bootstrap fresh heads. Training the *same* data from a raw DeBERTa-v3-base encoder (`from_encoder`, fresh heads, 15 epochs) instead of warm-starting collapses every span/relation task: strict micro-F1 entity 0.141, relation 0.000, event-trigger 0.221, event-argument 0.000 (0.168 relaxed), classification 0.356 — only coarse event-type (0.998, few classes) survives. So ~1.5K synthetic records are an *adaptation* corpus, not a from-scratch pretraining set: the extraction heads need either a warm start or the fastino curriculum's $\sim 10^5$–10^6 scale. This restates the head-init thesis (§10.5) from the data side. Checkpoint: `whr778/deberta-base-fromenc-synthetic`.

10.7 How much data warms the head? An mmBERT data-scaling curve

§10.6's light broad pass did not lift arguments, but it was confounded (2 epochs, NER-diluted, `eval_loss`-selected). This experiment isolates the one variable that matters — **Stage-A corpus size** — and measures it directly. We warm mmBERT-base fresh heads (`from_encoder`) on a structure/argument-dense event corpus of size N, then fine-tune each on RAMS under the **fixed** `mmbert-base-rams` recipe (15 epochs, `bce_posweight` 4.0, native long-context, argument-strict selection), and read RAMS blind-test argument-strict F1 vs N. The Stage-A corpus is assembled from the multilingual event corpora already on disk (docee, chfinann, docfee, duee, cmnee, maven, text2json, events_biotech, mendeley_ed, casie), nested and proportional at 10K/40K/100K, with RAMS/WikiEvents held out (leakage). Spec: `events_working_papers/PAPER_0_FOUNDATION.md sec 10.7`.

Two endpoints are already known on their respective encoders: **N=0 = 0.050** (mmBERT `from_encoder` straight to RAMS, §10.5); the DeBERTa-v3 fastino warm-start (254K) reaches **0.462** as a *cross-encoder reference*, not an mmBERT point (there is no fastino-scale warm-start on mmBERT). RAMS blind test (871 docs), strict micro-F1:

Stage-A size N	event_argument (S)	event_trigger (S)	event_type (S)
0 (fresh → RAMS)	0.050	0.611	—
10K	0.050	0.598	0.952
40K	0.115	0.706	0.931
~100K	**0.158**	0.732	0.949
254K (DeBERTa ref)	0.462	0.935	—

10K of head-warming moved the argument head essentially zero (0.050, identical to the N=0 floor), but **40K lifts it ~2.3× to 0.115** and **100K to 0.158** (trigger climbs 0.611 → 0.732 alongside). Two conclusions: (i) the **knee is between 10K and 40K** — mmBERT's argument head *does* warm from broad structure/argument data, at a higher data threshold than DeBERTa (confirming a cold multilingual encoder needs more of it); and (ii) the curve **keeps climbing through 100K with no plateau** — roughly log-linear, each ~2.5× of data adding ~0.04-0.05 argument-F1. So 100K is still well short of saturation: more structure/argument data would keep helping, though gradually, and matching the DeBERTa-v3 fastino warm-start (0.462) on mmBERT would take substantially more than 100K. The practical read for a shippable multilingual event model: warm mmBERT on a large structure/argument curriculum (≥40K, more is better), then fine-tune. Checkpoints (private): `whr778/scaling-mmbert-{10k,40k,100k}` and `-rams`.

11. Reproducibility

- **Train** from a raw backbone: `uv run python tools/train/train.py --config tools/train/config/<name>.yaml`. WikiEvents configs enable windowed global decoding at eval (`eval.global_decode: true`, `chunk_size: 384`, `chunk_overlap: 128`).
- **Convert** corpora with the scripts in `tools/data/` (see `run_all_converters.sh` and `TRAINING_DATA.md`).
- **Infer** at the document level: `tools/infer.py --model <ckpt> --input <doc> --events '{"Attack":["Attacker","Target","Place"]}' --global-decode`.
- Design and verification notes: `PAPER_0_FOUNDATION.md sec 9`, `METRICS.md`, `PROJECT_HISTORY.md`.

12. Limitations and future work

- We ran the long-context mmBERT experiment (§10.5) and the result overturned the expectation: a from-encoder mmBERT does **not** recover the argument bottleneck, because — as the DeBERTa-v3

encoder-isolation control shows — that bottleneck is **head initialization** (IE-curriculum pretraining), not the context window. Long context still remedies window-bound recall loss on genuinely long documents (its intended role, §9.5), but it is orthogonal to head competence. We ran a first, cheap version of the head-init A/B (§10.6): a 2-epoch, `eval_loss`-selected broad+synthetic base did **not** lift downstream WikiEvents events (arguments stayed at the floor; event_type regressed). The open work is therefore a *heavier* head-init pass — the broad-label, multi-task synthetic curriculum in `tools/data/synthetic/` at ~10^5–10^6 scale with **argument-strict checkpoint selection**, matching the fastino curriculum — rather than a light multi-task warm-up, then re-run RAMS/WikiEvents and A/B against the fastino heads.

- The global decoder's weights are **heuristic, not learned**: the beam optimizes one fixed objective — summed argument confidence minus a flat `conflict_penalty` per reused span, under hard trigger-floor and single-filler rules (`GlobalDecodeConfig`) — with no graph-level feature vector. A true OneIE-style scorer would add global features over the candidate graph (role co-occurrence, cross-event argument sharing, entity-type↔role compatibility) with weights fit by **structured-perceptron beam training** against gold document graphs; WikiEvents/RAMS supply the graphs, and the windowed candidates this module already emits are the training substrate. We implemented the lighter intermediate — `sweep_global_decode` fits the `GlobalDecodeConfig` scalars on validation, as `sweep_thresholds` fits the decision threshold (§8) — and measured it within noise on the WikiEvents test set (§10.2, row D), so the flat response is structural, not a tuning artifact; only the fully learned scorer remains open. Either way the gain is **precision, not recall**: the scorer only re-ranks candidates already generated, so it cannot recover an argument that fell outside every window — the dominant error term (§10.1: 231 FN vs. 123 FP) — which is why the beam is already neutral over the greedy merge (§10.2). Crucially, a long-context backbone does **not** retire this decoder: the windowed regime reappears on mmBERT whenever the *trained* `max_len` is narrower than the document — from genuine overflow (documents past 8192: common for book-length corpora, rare for WikiEvents) or from training at a reduced `max_len` for compute (8192-token attention is quadratic; §9.5). Long context shrinks the population where the decoder fires, it does not eliminate it; and as the window grows, more trigger↔argument pairs are already co-located within it, so the residual decode collapses toward the dedup the greedy merge already performs. The learned scorer is therefore the polish for the windowed regime whenever it is active — second-order to the within-window recall ceiling throughout.
- Long-range arguments beyond one window remain an upstream recall ceiling; trigger-anchored windows or a two-stage document-level argument reader are the natural next steps (retired RECOMMENDATIONS note, Options 2-3; see PROJECT_HISTORY.md).
- Argument coreference is not resolved in the span-architecture models reported here; document-level benchmarks score against any coreferential mention, so measured argument F1 in §10 is a lower bound. **Resolved on the boundary architecture in Paper 3 (`PAPER_3_DOCUMENT_EVENTS_AND_COREF.md` §4):** `record_coreferent_ownership: true`, `TriggerLink` (`record_coref_link: true` with 64-width MLP distance), `EntityCorefLink` (`entity_coref_link: true`), and occurrence-grouped coreferent argument targets (`expand_argument_coreferents`) now train and decode coreferent event and argument clusters directly.

- Similarly, the 263,949-document multi-task curriculum, trigger-seeded record individuation (`event_records: true`), relational junction layer (`record_link_mode: junction`), and split positive-negative supervision (`pair_split_reduction: true`) in `eb22g` complete the from-encoder mmBERT head-pretraining programme proposed in the first bullet above; see `PAPER_3_DOCUMENT_EVENTS_AND_COREF.md` §5–6.
- Packaging: several training-only dependencies and the raised Python floor should move to a `training` extra before any of this lands in the base inference package (see PROJECT_HISTORY.md (retired documents)).

13. References

Methods and models

- Zaratiana, U., Pasternak, G., Boyd, O., Hurn-Maloney, G., Lewis, A. (2025). *GLiNER2: Schema-Driven Multi-Task Learning for Structured Information Extraction.* EMNLP 2025 System Demonstrations, pp. 130–140. ACL. https://aclanthology.org/2025.emnlp-demos.10/
- Zaratiana, U., Tomeh, N., Holat, P., Charnois, T. (2024). *GLiNER: Generalist Model for Named Entity Recognition using Bidirectional Transformer.* NAACL. https://aclanthology.org/2024.naacl-long.300/
- Lin, Y., Ji, H., Huang, F., Wu, L. (2020). *A Joint Neural Model for Information Extraction with Global Features* (OneIE). ACL. https://aclanthology.org/2020.acl-main.713/
- Ortmann, K. (2022). *Fine-Grained Error Analysis and Fair Evaluation of Labeled Spans.* LREC. https://aclanthology.org/2022.lrec-1.150/
- Marone, M., Weller, O., Fleshman, W., Yang, E., Lawrie, D., Van Durme, B. (2025). *mmBERT: A Modern Multilingual Encoder with Annealed Language Learning.* arXiv:2509.06888. https://arxiv.org/abs/2509.06888
- Warner, B., et al. (2024). *Smarter, Better, Faster, Longer: A Modern Bidirectional Encoder for Fast, Memory Efficient, and Long Context Finetuning and Inference* (ModernBERT). arXiv:2412.13663. https://arxiv.org/abs/2412.13663
- He, P., Gao, J., Chen, W. (2021). *DeBERTaV3: Improving DeBERTa using ELECTRA-Style Pre-Training with Gradient-Disentangled Embedding Sharing.* arXiv:2111.09543. https://arxiv.org/abs/2111.09543
- Lin, T.-Y., Goyal, P., Girshick, R., He, K., Dollár, P. (2017). *Focal Loss for Dense Object Detection.* ICCV. https://arxiv.org/abs/1708.02002
- Ben-Baruch, E., et al. (2021). *Asymmetric Loss for Multi-Label Classification.* ICCV. https://arxiv.org/abs/2009.14119
- Li, X., Sun, X., Meng, Y., Liang, J., Wu, F., Li, J. (2020). *Dice Loss for Data-imbalanced NLP Tasks.* ACL. https://arxiv.org/abs/1911.02855

Datasets

Datasets used for training/evaluation on this branch (full corpus list and provenance in `tools/data/TRAINING_DATA.md`):

- Li, S., Ji, H., Han, J. (2021). *Document-Level Event Argument Extraction by Conditional Generation* (WikiEvents / BART-Gen). NAACL.
- Walker, C., Strassel, S., Medero, J., Maeda, K. (2006). *ACE 2005 Multilingual Training Corpus.* LDC2006T06, Linguistic Data Consortium.
- Tong, M., et al. (2022). *DocEE: A Large-Scale and Fine-grained Benchmark for Document-level Event Extraction.* NAACL. https://aclanthology.org/2022.naacl-main.291/
- Ebner, S., Xia, P., Culkin, R., Rawlins, K., Van Durme, B. (2020). *Multi-Sentence Argument Linking* (RAMS). ACL. https://aclanthology.org/2020.acl-main.718/
- Wang, X., et al. (2020). *MAVEN: A Massive General Domain Event Detection Dataset.* EMNLP. https://aclanthology.org/2020.emnlp-main.129/
- Satyapanich, T., Ferraro, F., Finin, T. (2020). *CASIE: Extracting Cybersecurity Event Information from Text.* AAAI.
- Zhu, M., et al. (2024). *CMNEE: A Large-Scale Document-Level Event Extraction Dataset based on Open-Source Chinese Military News.* LREC-COLING. https://aclanthology.org/2024.lrec-main.299/
- Bogdanov, S., et al. (2024). *NuNER: Entity Recognition Encoder Pre-training via LLM-Annotated Data.*
- Zhou, W., et al. (2024). *UniversalNER: Targeted Distillation from Large Language Models for Open Named Entity Recognition* (Pile-NER). ICLR.
- Yao, Y., et al. (2019). *DocRED: A Large-Scale Document-Level Relation Extraction Dataset.* ACL.
- Tan, Q., Xia, L., Bing, L. (2022). *Revisiting DocRED — Addressing the False Negative Problem in Relation Extraction* (Re-DocRED). EMNLP.
- Luan, Y., He, L., Ostendorf, M., Hajishirzi, H. (2018). *Multi-Task Identification of Entities, Relations, and Coreference for Scientific Knowledge Graph Construction* (SciERC). EMNLP.
- Luo, L., et al. (2022). *BioRED: A Rich Biomedical Relation Extraction Dataset.* Briefings in Bioinformatics.
- Park, S., et al. (2021). *KLUE: Korean Language Understanding Evaluation.* NeurIPS Datasets and Benchmarks.
- Adelani, D.I., et al. (2022). *MasakhaNER 2.0: Africa-centric Transfer Learning for Named Entity Recognition.* EMNLP. https://aclanthology.org/2022.emnlp-main.298/
- Adelani, D.I., et al. (2023). *MasakhaNEWS: News Topic Classification for African Languages.* IJCNLP-AACL. https://aclanthology.org/2023.ijcnlp-main.10/
- Pan, X., Zhang, B., May, J., Nothman, J., Knight, K., Ji, H. (2017). *Cross-lingual Name Tagging and Linking for 282 Languages* (WikiANN). ACL. https://aclanthology.org/P17-1178/
- Rahimi, A., Li, Y., Cohn, T. (2019). *Massively Multilingual Transfer for NER* (PAN-X balanced splits). ACL. https://aclanthology.org/P19-1015/

Appendix A: module map

Area	Location
Events (schema/engine/api/data)	`gliner2/inference/schema.py`, `engine.py`, `runtime.py` (extract methods), `gliner2/api_client.py`, `training/data.py`
Encoder-agnostic loading, losses	`gliner2/models/span/model.py` + `gliner2/configuration.py` (re-exported via the `gliner2/model.py` shim)
Training infrastructure	`gliner2/training/trainer.py`, `parallel.py`, `eta.py`, `chunking.py`
Metrics + fair error analysis	`gliner2/training/eval_metrics.py` (re-exported via `metrics.py`), `METRICS.md`
Global event decoder	`gliner2/inference/global_decode.py`, `chunking.py`
Config-driven training	`tools/train/train.py`, `tools/train/config/`
Data converters	`tools/data/convert_*.py`
Inference CLI	`tools/infer.py`

Appendix B: scope

The work described here spans two architectures. §1–§10.4 describe the **span** architecture (DeBERTa-v3, `fastino/gliner2-base-v1`); §10.5–10.7 use the same training and evaluation infrastructure on the long-context mmBERT backbone. Core-package changes are catalogued in `PROJECT_HISTORY.md`; the training tooling in `tools/train/` and `tools/data/` is net-new.

Appendix C: Running the models — Hub download and the viewer

The fine-tuned checkpoints are published as Hub repos `whr778/<config>` (e.g. `whr778/gliner2-large-v1-docee`). **Most are PRIVATE — 28 of the 53 `whr778/*` entries in the viewer registry — so a Hugging Face token is required or they fail with a 401.** Read scope is enough:

```
export HF_TOKEN=hf_...        # or: huggingface-cli login
```

Two steps to obtain and run them.

Download from the Hub. `scripts/pull_from_hf.sh` fetches each model into `out/fastino/<config>/best/` (where the viewer auto-discovers it) via `hf download` with Xet high-performance transfer — parallel and CDN-accelerated, so far faster than a single rsync-over-SSH stream on a high-latency link:

```
bash scripts/pull_from_hf.sh                        # all trained models
bash scripts/pull_from_hf.sh gliner2-large-v1-docee  # or specific ones
```

Launch the viewer. An interactive app (NextJS + FastAPI) that runs any local checkpoint over pasted / uploaded / URL-imported text and renders entities, relations, events, classifications, and structures with confidences:

```
cd viewer/frontend && npm install     # one-time
bash viewer/viewer.sh start           # backend :8000 + frontend :3000 (waits until up)
# open http://localhost:3000 ; stop both with: bash viewer/viewer.sh stop
```

In the browser, pick a model in the **Model** box (downloaded checkpoints are listed automatically) — its training-time schema loads to match — then enter text and click **Extract**. Presets include the EKF pipeline's own two schemas, **Casualty report (stage 2)** and **Relevance gate (stage 0)**, so the viewer can reproduce what each stage actually asks of a model.

A structure schema needs `mode` and `anchor` on boundary models. Without them the record head cannot decode and extraction returns `{}` with no error — which reads as "the model found nothing" rather than "this schema was undecodable". The viewer backend fills them in when omitted; a schema copied elsewhere must declare them itself. Span models ignore the metadata, so declaring it costs nothing. Set a default model at launch with `GLINER2_MODEL=out/fastino/gliner2-large-v1-docee/best bash viewer/viewer.sh start`.

Or run headless at the document level from the CLI:

```
uv run python tools/infer.py --model out/fastino/gliner2-large-v1-docee/best \
  --input <doc.txt> --events '{"Attack":["Attacker","Target","Place"]}' --global-decode
```